Day 1A — Tokens, Tokenization & Embeddings

> 🎯 **Goal**: Understand how LLMs represent language — from raw text to the numerical vectors that make reasoning possible.

> ⏱️ **Time**: 1h theory + 0.5h practice

> 📦 **Deliverable**: Python notebook demonstrating tokenization with tiktoken + embedding similarity calculations

---

Table of Contents

1. Why This Matters
2. Tokens & Tokenization
3. Embeddings — The Missing Foundation
4. Putting It Together
5. Practice Exercises

Why This Matters

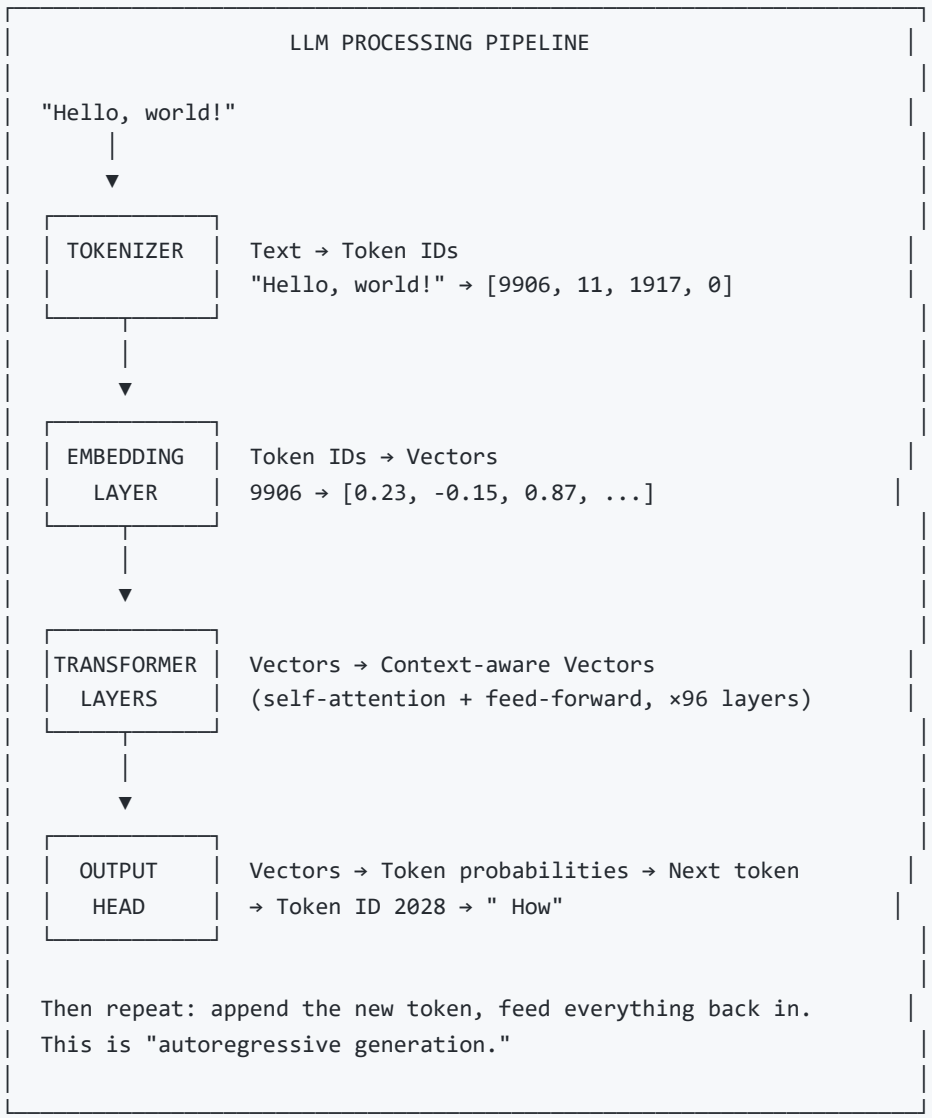
LLMs are **next-token predictors**, not magic. Every time GPT-5 or Claude Opus writes a sentence, it is doing one thing: predicting the most likely next token given everything that came before. That's it.

Why should you care about the internals? Because when you build agents, **you will hit problems** that only make sense if you understand the machinery underneath:

- **Token limits**: Why does your agent crash with long documents? Tokens.
- **Cost estimation**: API pricing is per-token, not per-word.
- **Prompt engineering**: Some prompts work better because of how tokenization splits the text.
- **Retrieval (RAG)**: Embeddings power the "find similar documents" step.
- **Debugging**: When the model produces garbage, understanding tokens helps you see why.

The Full Pipeline

Every time you send text to an LLM, this is what actually happens:



Today we focus on the first two boxes: **Tokenizer** and **Embedding Layer**. Parts B and C will cover the Transformer itself.

Tokens & Tokenization

What Is a Token?

A token is **NOT a character** and **NOT a word**. It is a **subword unit** — a chunk of text that the model treats as a single atomic symbol.

Why not just use characters? Too granular — the model would need to "figure out" spelling before it could reason about meaning. Why not whole words? Too many unique words in human language (think: "unhappiness", "unhappy", "happiness", "happily" — do we need separate entries for all of them?).

Subword tokenization is the sweet spot: common words get their own token ("the", "hello"), while rare words get split into reusable pieces ("un" + "happi" + "ness").

```
┌──────────────────────────────────────────────────────────┐
│               WHY SUBWORD TOKENIZATION?                  │
│                                                          │
│  Characters:  H | e | l | l | o                          │
│               Too granular — model must learn spelling   │
│                                                          │
│  Words:       Hello | , | world | !                      │
│               Vocabulary explodes — what about "unhappily"?  │
│                                                          │
│  Subwords:    Hello | , |  world | !         ← GPT-5 style  │
│               Best of both worlds — finite vocab, any text   │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Why Tokenization Exists

The core problem: human language is **infinite** (you can always write a new sentence), but a neural network needs a **finite** set of input symbols. Tokenization is the bridge — it maps infinite text to a fixed vocabulary of ~100k tokens.

BPE: Byte Pair Encoding

The most common tokenization algorithm (used by the GPT series, Claude, and most modern LLMs) is **Byte Pair Encoding**. Here's how it works, step by step:

Starting point: Every individual byte (character) is a token.

Algorithm: Repeatedly find the most frequent pair of adjacent tokens and merge them into a new token. Stop when you reach your desired vocabulary size.

```
BPE Walkthrough — Training on: "low lower lowest"

Step 0 (character-level):
  l o w _ l o w e r _ l o w e s t
  (where _ = end-of-word marker)

Step 1: Most frequent pair = (l, o) → merge into "lo"
  lo w _ lo w e r _ lo w e s t

Step 2: Most frequent pair = (lo, w) → merge into "low"
  low _ low e r _ low e s t

Step 3: Most frequent pair = (e, r) appears once, (e, s) appears once,
         (low, _) appears once... pick (low, e) → merge into "lowe"
  low _ lowe r _ lowe s t
```

```
...and so on until vocabulary size is reached.

Result: The tokenizer learns that "low" is a reusable unit!
```

Token ≠ Word: The Pitfalls

This is where things get surprising. Tokenization creates **counter-intuitive splits** that directly affect model behavior:

1. Leading whitespace becomes part of the token

```
# " world" (with leading space) is ONE token
# "world" (without space) is a DIFFERENT token
# This is why formatting matters in prompts!
```

2. Long words get split into multiple tokens

```
"tokenization"  →  ["token", "ization"]       (2 tokens)
"embedding"     →  ["embed", "ding"]           (2 tokens)
"transformer"   →  ["trans", "former"]         (2 tokens)
"antidisestablishmentarianism" → 5+ tokens
```

3. Numbers are tokenized weirdly

```
"123"    →  ["123"]                  (1 token — small number, common)
"123456" →  ["123", "456"]           (2 tokens — split at 3 digits)
"98765"  →  ["987", "65"]            (2 tokens — inconsistent split!)

This is why LLMs are bad at arithmetic — the digits are jumbled across tokens!
```

4. Multilingual text is expensive

```
English: "Hello"        → 1 token
Chinese: "你好"          → 2-3 tokens (each character may be a token)
Arabic:  "5-3 →         "مرحبا tokens
Korean:  "안녕하세요"      → 3-5 tokens

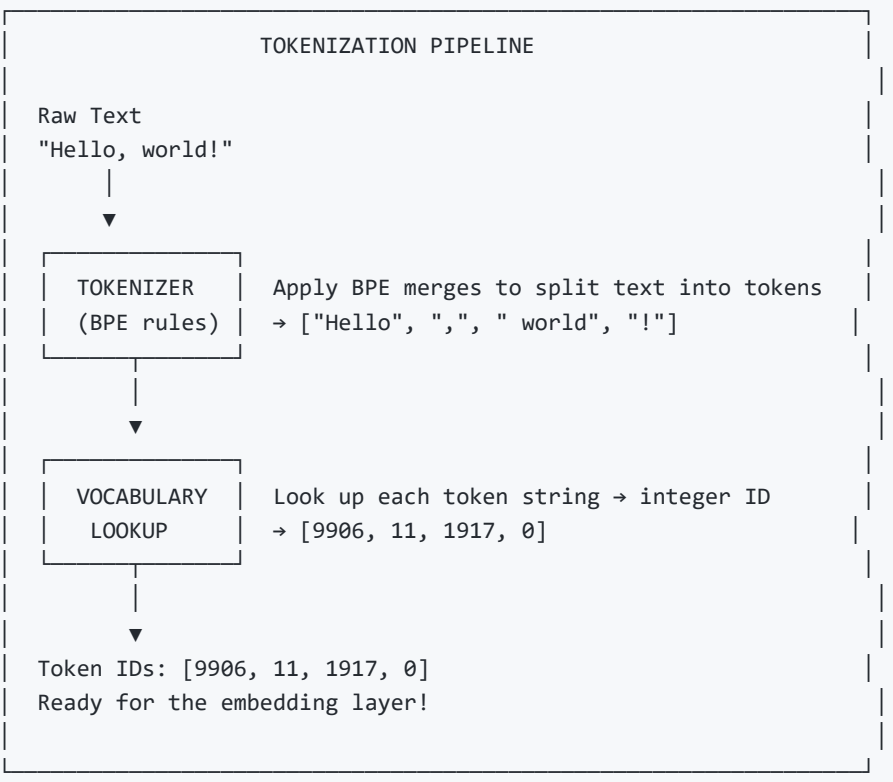
Same meaning, wildly different token counts. English dominates the training
data, so English gets more efficient tokenization.
```

Vocabulary Sizes

```
┌──────────────────────────┬──────────────────┬──────────────────────────────┐
│ Model                    │ Vocab Size       │ Tokenizer                    │
├──────────────────────────┼──────────────────┼──────────────────────────────┤
│ GPT-5 / GPT-5.1 / 5.2    │ ~200,000         │ o200k_base                   │
│ GPT-4o                   │ ~200,064         │ o200k_base                   │
│ GPT-4                    │ ~100,256         │ cl100k_base                  │
│ Claude (Opus 4.6,        │ ~100,000+        │ BPE (proprietary)            │
│  Sonnet 4.5)             │                  │                              │
│ Gemini 3 Pro             │ ~256,000         │ SentencePiece                │
│ LLaMA 3 / 4              │ ~128,256         │ tiktoken-based BPE           │
│ LLaMA 2                  │ ~32,000          │ SentencePiece BPE            │
│ GPT-2                    │ ~50,257          │ BPE (original)               │
│ BERT                     │ ~30,522          │ WordPiece                    │
└──────────────────────────┴──────────────────┴──────────────────────────────┘

Larger vocabulary = fewer tokens per text = faster inference
  but also = larger embedding matrix = more memory
```

The Tokenization Pipeline



Code Example: Tokenization with tiktoken

```python
import tiktoken

# Load the tokenizer (o200k_base, used by GPT-4o and GPT-5 series)
```

```
enc = tiktoken.encoding_for_model("gpt-4")

# Encode text into token IDs
text = "Hello, world!"
tokens = enc.encode(text)
print(f"Token IDs: {tokens}")
# Token IDs: [9906, 11, 1917, 0]

# Decode each token individually to see what it represents
token_strings = [enc.decode([t]) for t in tokens]
print(f"Token strings: {token_strings}")
# Token strings: ['Hello', ',', ' world', '!']

# Count tokens (critical for API cost estimation)
print(f"Token count: {len(tokens)}")
# Token count: 4

# Demonstrate the pitfalls
examples = {
    "tokenization":  enc.encode("tokenization"),
    "123456":        enc.encode("123456"),
    "Hello":         enc.encode("Hello"),
    " Hello":        enc.encode(" Hello"),   # note the leading space
}

for text, toks in examples.items():
    decoded = [enc.decode([t]) for t in toks]
    print(f"  '{text}' → {len(toks)} tokens: {decoded}")

# Output:
#   'tokenization' → 2 tokens: ['token', 'ization']
#   '123456' → 2 tokens: ['123', '456']
#   'Hello' → 1 tokens: ['Hello']
#   ' Hello' → 1 tokens: [' Hello']    ← space is part of the token!
```

Embeddings — The Missing Foundation

Why We Need Embeddings

We now have token IDs — integers like 9906 for "Hello" and 1917 for " world". But there's a problem: **these numbers are arbitrary**. The fact that 9906 comes before 9907 means nothing about meaning. "Hello" (9906) is not more similar to token 9907 than to token 42.

We need a representation where **meaning is captured numerically** — where similar concepts have similar numbers.

That's what embeddings do.

Analogy: GPS Coordinates for Meaning

Think of embeddings as **GPS coordinates, but for meaning instead of geography**.

```
GPS maps physical locations to coordinates:
  New York  → (40.7, -74.0)
  London    → (51.5, -0.1)
  Paris     → (48.9,  2.3)

  London and Paris are geographically close → their coordinates are close.
  New York is far from both → its coordinates are far.

Embeddings map MEANING to coordinates:
  "king"    → [0.82, -0.31, 0.55, 0.12, ...]    (768 dimensions)
  "queen"   → [0.79, -0.28, 0.53, 0.14, ...]    (nearby!)
  "banana"  → [-0.45, 0.67, -0.22, 0.88, ...]   (far away)

  "king" and "queen" mean related things → their vectors are close.
  "banana" is semantically unrelated → its vector is distant.
```

How Embedding Space Works

Embeddings live in **high-dimensional space** — typically 768 to 4096+ dimensions. We can't visualize 768 dimensions, but the math works the same as in 2D or 3D:

```
Key properties of embedding space:

  1. DIMENSIONS: Each of the 768+ dimensions captures some aspect
     of meaning. No single dimension means "is-a-verb" or
     "is-positive" — it's distributed across many dimensions.

  2. DISTANCE = SIMILARITY: Words with similar meanings are
     nearby in this space (measured by cosine similarity).

  3. DIRECTIONS HAVE MEANING: The direction from "man" to "woman"
     captures the concept of gender. The direction from "walk" to
     "walked" captures past tense.
```

King − Man + Woman = Queen

This famous example demonstrates that embedding space captures **relational structure**, not just similarity. Here's why it works:

```
┌──────────────────────────────────────────────────────────────┐
│              VECTOR ARITHMETIC IN EMBEDDING SPACE            │
│                                                              │
│  The direction from "man" → "woman" encodes a gender shift.  │
│  The direction from "king" → ??? should be the same shift.   │
│  So king − man + woman = ... queen!                          │
│                                                              │
│  Vector math:                                                │
```

```
│    king  = [0.82, -0.31, 0.55, 0.12]                                  │
│  - man   = [0.60, -0.20, 0.48, 0.35]                                  │
│  + woman = [0.58, -0.17, 0.46, 0.38]                                  │
│  ──────────────────────────────────────                               │
│  result  = [0.80, -0.28, 0.53, 0.15]                                  │
│                                                                       │
│  Nearest neighbor of result → "queen" [0.79, -0.28, 0.53, 0.14]       │
│                                                                       │
│                                                                       │
│  Simplified 2D visualization:                                         │
│                                                                       │
│       queen ♛ ·                                                       │
│               ↑  gender                                               │
│       king  ♚ ·  direction                                            │
│                                                                       │
│                     woman ♀ ·                                         │
│                             ↑  gender                                 │
│                     man   ♂ ·  direction                              │
│                                                                       │
│  Same offset (gender direction) applied to "king" lands on            │
│  "queen". The space has LEARNED these relationships.                  │
│                                                                       │
└───────────────────────────────────────────────────────────────────────┘
```

Why does this matter for agents? Because this same principle powers:

- **RAG**: Finding relevant documents by vector similarity
- **Classification**: Grouping similar inputs together
- **Tool selection**: Matching user intent to the right tool

Token Embeddings vs Contextual Embeddings

This is a critical distinction that many people miss:

```
┌───────────────────────────────────────────────────────────────────────┐
│        TOKEN EMBEDDINGS vs CONTEXTUAL EMBEDDINGS                      │
│                                                                       │
│  TOKEN EMBEDDING (before Transformer):                                │
│  ──────────────────────────────────────                               │
│  A static lookup table. The word "bank" ALWAYS gets the same          │
│  vector, regardless of context.                                       │
│                                                                       │
│    "river bank"  → bank → [0.23, -0.15, 0.87, ...]                    │
│    "bank account" → bank → [0.23, -0.15, 0.87, ...]  ← SAME!          │
│                                                                       │
│  Problem: "bank" means completely different things in these           │
│  sentences, but gets the same representation.                         │
│                                                                       │
│                                                                       │
│  CONTEXTUAL EMBEDDING (after Transformer):                            │
│  ──────────────────────────────────────────                           │
│  After passing through attention layers, each token's vector          │
```

```
│  has been enriched with information from ALL other tokens.       │
│                                                                  │
│    "river bank"   → bank → [0.85, -0.40, 0.12, ...]              │
│    "bank account" → bank → [-0.33, 0.72, 0.55, ...]  ← DIFF!     │
│                                                                  │
│  Now the representation captures the MEANING IN CONTEXT.         │
│                                                                  │
│  This is exactly WHY we need the Transformer — to transform      │
│  static token embeddings into context-aware representations.     │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Embedding Space Visualization

If we squash the 768+ dimensions down to 2D (using techniques like t-SNE or UMAP), we can see that **semantically related words cluster together**:

```
┌──────────────────────────────────────────────────────────────────┐
│          2D PROJECTION OF EMBEDDING SPACE                        │
│                                                                  │
│                                                                  │
│        cat ·   · dog                                             │
│     kitten ·      · puppy            · red                       │
│      tiger ·         · wolf          · blue                      │
│                                      · green                     │
│                                      · yellow                    │
│                                                                  │
│                                                                  │
│                  · happy                                         │
│                  · joyful     · Paris                            │
│                  · glad       · London                           │
│                               · Tokyo                            │
│       · sad                   · Berlin                           │
│       · gloomy                                                   │
│       · melancholy                                               │
│                                                                  │
│                                                                  │
│  Animals cluster together.  Colors cluster together.             │
│  Emotions cluster together. Cities cluster together.             │
│  Antonyms (happy/sad) are related but separated.                 │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Putting It Together

The Embedding Matrix

In practice, the embedding layer is a **giant lookup table** — a matrix of shape `vocabulary_size × embedding_dimension`:

```
┌──────────────────────────────────────────────────────────────────┐
│                  THE EMBEDDING MATRIX                            │
│                                                                  │
│  Vocabulary size: 100,256 tokens                                 │
│  Embedding dim:   4,096–16,384 (varies by model)                 │
│                                                                  │
│  Matrix shape: 100,256 × 4,096                                   │
│  That's ~410 million parameters JUST for the embedding layer!    │
│                                                                  │
│         dim 0   dim 1   dim 2   dim 3   ...   dim 4095           │
│        ┌───────┬───────┬───────┬───────┬─────┬────────┐          │
│  ID 0  │  0.12 │ -0.34 │  0.56 │  0.01 │ ... │  0.23  │ "!"      │
│  ID 1  │ -0.05 │  0.22 │  0.18 │ -0.67 │ ... │  0.44  │ ...      │
│  ...   │  ...  │  ...  │  ...  │  ...  │ ... │  ...   │ ...      │
│  9906  │  0.82 │ -0.31 │  0.55 │  0.12 │ ... │ -0.09  │ "Hello"  │
│  ...   │  ...  │  ...  │  ...  │  ...  │ ... │  ...   │ ...      │
│        └───────┴───────┴───────┴───────┴─────┴────────┘          │
│                                                                  │
│  To embed token 9906 ("Hello"):                                  │
│    → Just look up row 9906 → [0.82, -0.31, 0.55, 0.12, ...]      │
│                                                                  │
│  That's it! Each token ID is an INDEX into this matrix.          │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

These Embeddings Are Learned

A crucial point: nobody hand-designs these vectors. The embedding matrix is **initialized randomly** and then **updated during training** alongside all the other model parameters. Through billions of training examples, the model discovers that certain arrangements of numbers (certain regions of embedding space) are useful for predicting the next token.

The king/queen relationship, the clustering of animals, the separation of topics — all of this **emerges from training**, not from human engineering.

What Happens After Embedding?

Once each token has its embedding vector, two more things are added before the Transformer layers:

1. **Positional information** — the embedding alone doesn't know if "bank" is the 1st or 50th word. Position encodings are added so the model knows word order. (Covered in Day 1C)
2. **The vectors enter the Transformer** — where self-attention creates the contextual embeddings we discussed above. (Covered in Day 1B)

Practice Exercises

Exercise 1: Token Exploration

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4")

# 1. Tokenize these sentences and compare token counts
sentences = [
    "The quick brown fox jumps over the lazy dog.",
    "Supercalifragilisticexpialidocious is a long word.",
    "こんにちは世界",                          # Japanese "Hello World"
    "2 + 2 = 4",
    "2222222 + 3333333 = 5555555",
    "def fibonacci(n): return n if n <= 1 else fibonacci(n-1) + fibonacci(n-2)",
]

for s in sentences:
    tokens = enc.encode(s)
    decoded = [enc.decode([t]) for t in tokens]
    print(f"\n'{s}'")
    print(f"  Tokens ({len(tokens)}): {decoded}")
```

Exercise 2: Cost Estimation

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4")

def estimate_cost(text, model="gpt-4", is_input=True):
    """Estimate the API cost of a text string."""
    tokens = enc.encode(text)
    n = len(tokens)

    # Approximate pricing (USD per 1M tokens, check provider for current rates)
    rates = {
        "gpt-5.2":       {"input": 2.00,  "output": 8.00},
        "gpt-5.1":       {"input": 1.00,  "output": 4.00},
        "gpt-4o":        {"input": 2.50,  "output": 10.00},
        "claude-opus":   {"input": 15.00, "output": 75.00},
        "claude-sonnet": {"input": 3.00,  "output": 15.00},
    }

    rate = rates[model]["input" if is_input else "output"]
    cost = (n / 1_000_000) * rate

    print(f"Text: '{text[:50]}...'")
```

```
    print(f"  Tokens: {n}")
    print(f"  Estimated cost ({model}, {'input' if is_input else 'output'}): ${cost:
    return cost

# Try it with a system prompt
system_prompt = """You are an expert code reviewer. Analyze the following code
for bugs, security vulnerabilities, and performance issues. For each issue found,
provide the line number, severity (high/medium/low), and a suggested fix."""

estimate_cost(system_prompt, "gpt-5.1")
estimate_cost(system_prompt, "claude-sonnet")
```

Exercise 3: Tokenization Gotchas Quiz

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4")

# Predict the token count BEFORE running each one:
quiz = [
    ("Hello", "?"),           # Guess: ___
    (" Hello", "?"),          # Guess: ___ (note: leading space)
    ("hello", "?"),           # Guess: ___ (note: lowercase)
    ("HELLO", "?"),           # Guess: ___
    ("   ", "?"),             # Guess: ___ (three spaces)
    ("100", "?"),             # Guess: ___
    ("1000000", "?"),         # Guess: ___
    ("🎉", "?"),               # Guess: ___
]

print("Tokenization Quiz — were your guesses right?\n")
for text, _ in quiz:
    tokens = enc.encode(text)
    decoded = [enc.decode([t]) for t in tokens]
    print(f"  '{text}' → {len(tokens)} token(s): {decoded}")
```

✓ Day 1A Deliverable Checklist

- [ ] Can explain: What is a token and why not use characters or words?
- [ ] Can explain: How does BPE build a vocabulary from text?
- [ ] Can explain: Why are LLMs bad at arithmetic? (token splits)
- [ ] Can explain: What is an embedding and why are token IDs insufficient?
- [ ] Can explain: Token embedding vs contextual embedding — what's the difference?
- [ ] Ran tiktoken code and explored tokenization pitfalls

- [ ] Estimated API cost for a sample prompt
- [ ] Can explain: What does "king − man + woman = queen" demonstrate?

▶ 📝 **Answer Key (click to expand)**

Next → **Day 1B: The Transformer & Self-Attention**